

Writing Clean Python for Analytics - Best Practices

Transform Messy Scripts into Production-Ready Code

Video Title: *7 Python Best Practices Every Data Analyst Should Know*

Why Clean Code Matters?

- Messy code = bugs, maintenance nightmares, and collaboration hell
- "Works on my machine" is not acceptable in production
- Clean code is not about being fancy - it's about being maintainable

What You'll Learn:

1. **Naming Conventions** - Variables and functions that explain themselves
2. **Documentation** - Docstrings and comments (when to use each)
3. **Type Hints** - Catch bugs before runtime
4. **Function Design** - Single Responsibility Principle
5. **List Comprehensions** - Pythonic and fast
6. **Error Handling** - Fail gracefully, not silently
7. **DRY Principle** - Don't Repeat Yourself

The Cost of Messy Code:

- 80% of time spent reading code, 20% writing
- Technical debt compounds - refactoring gets harder
- Your future self will thank you (or curse you)

```
In [1]: import pandas as pd
import numpy as np
from typing import List, Dict, Optional, Tuple, Union
from pathlib import Path
import warnings

warnings.filterwarnings('ignore')
np.random.seed(42)
```

0. The Problem: Messy Analytics Code (Before)

```
In [2]: print("THE PROBLEM: MESSY ANALYTICS CODE")
print("="*70)

print("\n❌ EXAMPLE OF BAD CODE:")
print("""
```

```

# Bad: Cryptic names, no documentation, no error handling
def f(d, c, t):
    r = d[d[c] > t]
    return r

# What does this do? No idea!
df = pd.read_csv('data.csv')
x = f(df, 'sales', 1000)
y = x['sales'].mean()
print(y)

# More bad patterns:
def process_data(df):
    # Does 10 different things
    df = df.dropna()
    df['new_col'] = df['a'] * df['b']
    df = df[df['sales'] > 0]
    df = df.groupby('category').sum()
    df = df.sort_values('sales')
    # ... 50 more lines
    return df

# Magic numbers everywhere
if sales > 5000 and discount < 0.15:
    category = 'premium'
)

print("\n🤖 PROBLEMS:")
print("    1. Cryptic variable names (d, c, t, x, y)")
print("    2. No documentation – what does this do?")
print("    3. No type hints – what types expected?")
print("    4. God functions – do too many things")
print("    5. Magic numbers – what is 5000? 0.15?")
print("    6. No error handling – crashes on bad input")
print("    7. Not reusable – hard to test and maintain")

print("\n💡 Solution: Apply clean code principles!")
print("=*70)

```

THE PROBLEM: MESSY ANALYTICS CODE

❌ EXAMPLE OF BAD CODE:

```
# Bad: Cryptic names, no documentation, no error handling
def f(d, c, t):
    r = d[d[c] > t]
    return r

# What does this do? No idea!
df = pd.read_csv('data.csv')
x = f(df, 'sales', 1000)
y = x['sales'].mean()
print(y)

# More bad patterns:
def process_data(df):
    # Does 10 different things
    df = df.dropna()
    df['new_col'] = df['a'] * df['b']
    df = df[df['sales'] > 0]
    df = df.groupby('category').sum()
    df = df.sort_values('sales')
    # ... 50 more lines
    return df

# Magic numbers everywhere
if sales > 5000 and discount < 0.15:
    category = 'premium'
```

🤖 PROBLEMS:

1. Cryptic variable names (d, c, t, x, y)
2. No documentation – what does this do?
3. No type hints – what types expected?
4. God functions – do too many things
5. Magic numbers – what is 5000? 0.15?
6. No error handling – crashes on bad input
7. Not reusable – hard to test and maintain

💡 Solution: Apply clean code principles!

1. Practice #1: Naming Conventions - Self-Documenting Code

```
In [3]: print("\nPRACTICE #1: NAMING CONVENTIONS")
        print("="*70)

        print("\n❌ BAD NAMES:")
        print("""
        # Cryptic
        d = pd.read_csv('sales.csv')
```

```

x = d['amount'].sum()
t = 5000

# Ambiguous
data = get_data() # What data?
process()          # Process what?
temp = df.copy()   # Temporary what?

# Inconsistent
userAge = 25       # camelCase
user_name = 'Bob'  # snake_case
USERSCORE = 100    # UPPERCASE
""")

print("\n✅ GOOD NAMES:")
print("""
# Descriptive and clear
sales_df = pd.read_csv('sales.csv')
total_revenue = sales_df['amount'].sum()
premium_threshold = 5000

# Specific and actionable
customer_data = get_customer_data()
process_monthly_sales()
filtered_df = df.copy() # Or: high_value_customers

# Consistent (PEP 8)
user_age = 25           # Variables: snake_case
user_name = 'Bob'
MAX_RETRIES = 3         # Constants: UPPER_CASE
""")

# Practical examples
print("\n🔗 PRACTICAL EXAMPLES:")
print("""

# Bad
def calc(d, x, y):
    return d[x] * d[y]

# Good
def calculate_revenue(sales_df: pd.DataFrame,
                      price_col: str,
                      quantity_col: str) -> pd.Series:
    """Calculate revenue by multiplying price and quantity."""
    return sales_df[price_col] * sales_df[quantity_col]

print("# Bad: calc(d, x, y)")
print("# Good: calculate_revenue(sales_df, price_col, quantity_col)")
print("""
print("Which one is self-explanatory? ✓")

print("\n📋 NAMING RULES (PEP 8):")
print("    • Variables/Functions: snake_case (user_age, get_data)")
print("    • Classes: PascalCase (DataProcessor, SalesAnalyzer)")
print("    • Constants: UPPER_CASE (MAX_RETRIES, API_KEY)""")

```

```
print("    • Private: _leading_underscore (_internal_method)")
print("    • Dunder: __double_underscore__ (__init__, __str__)")

print("\n💡 PRO TIPS:")
print("    1. Use verbs for functions: get_, calculate_, process_, validate_"
print("    2. Use nouns for variables: user_data, total_sales, error_count")
print("    3. Boolean variables: is_, has_, can_, should_"
print("        Example: is_valid, has_permission, can_process")
print("    4. Avoid single letters except: i, j, k in loops, x, y for coords"
print("    5. Be specific: 'customer_purchase_history' > 'data'")
print("=*70)
```

PRACTICE #1: NAMING CONVENTIONS

❌ BAD NAMES:

```
# Cryptic
d = pd.read_csv('sales.csv')
x = d['amount'].sum()
t = 5000

# Ambiguous
data = get_data() # What data?
process()          # Process what?
temp = df.copy()   # Temporary what?

# Inconsistent
userAge = 25       # camelCase
user_name = 'Bob'  # snake_case
USERSCORE = 100    # UPPERCASE
```

✅ GOOD NAMES:

```
# Descriptive and clear
sales_df = pd.read_csv('sales.csv')
total_revenue = sales_df['amount'].sum()
premium_threshold = 5000

# Specific and actionable
customer_data = get_customer_data()
process_monthly_sales()
filtered_df = df.copy() # Or: high_value_customers

# Consistent (PEP 8)
user_age = 25           # Variables: snake_case
user_name = 'Bob'
MAX_RETRIES = 3         # Constants: UPPERCASE
```

🔗 PRACTICAL EXAMPLES:

```
# Bad: calc(d, x, y)
# Good: calculate_revenue(sales_df, price_col, quantity_col)
```

Which one is self-explanatory? ✓

📁 NAMING RULES (PEP 8):

- Variables/Functions: snake_case (user_age, get_data)
- Classes: PascalCase (DataProcessor, SalesAnalyzer)
- Constants: UPPERCASE (MAX_RETRIES, API_KEY)
- Private: `_leading_underscore` (`_internal_method`)
- Dunder: `__double_underscore__` (`__init__`, `__str__`)

💡 PRO TIPS:

1. Use verbs for functions: `get_`, `calculate_`, `process_`, `validate_`
2. Use nouns for variables: `user_data`, `total_sales`, `error_count`

3. Boolean variables: `is_`, `has_`, `can_`, `should_`
Example: `is_valid`, `has_permission`, `can_process`
 4. Avoid single letters except: `i`, `j`, `k` in loops, `x`, `y` for coords
 5. Be specific: `'customer_purchase_history' > 'data'`
- =====

2. Practice #2: Documentation - Docstrings and Comments

```
In [4]: print("\nPRACTICE #2: DOCUMENTATION")
print("="*70)

print("\n❌ BAD DOCUMENTATION:")
print("""
# Bad: Obvious comments (noise)
x = x + 1 # Increment x

# Bad: Outdated comments
# Calculate total (but actually calculates average now)
avg = sum(values) / len(values)

# Bad: No documentation
def process(df, threshold, mode):
    # What does this do? What are valid modes?
    pass
""")

print("\n✅ GOOD DOCUMENTATION:")
print("""

def filter_high_value_customers(
    df: pd.DataFrame,
    revenue_threshold: float = 10000.0,
    time_period: str = 'last_year'
) -> pd.DataFrame:
    """
    Filter customers with revenue above threshold in specified period.

    Args:
        df: DataFrame with columns ['customer_id', 'revenue', 'date']
        revenue_threshold: Minimum revenue to qualify (default: 10000.0)
        time_period: One of ['last_month', 'last_year', 'all_time']

    Returns:
        Filtered DataFrame with high-value customers

    Raises:
        ValueError: If time_period is invalid
        KeyError: If required columns missing

    Example:
    >>> high_value = filter_high_value_customers(
    ...     df, revenue_threshold=50000, time_period='last_year'
    ... )
    """
    """
```

```

"""
# Validate inputs
required_cols = ['customer_id', 'revenue', 'date']
if not all(col in df.columns for col in required_cols):
    raise KeyError(f"DataFrame must have columns: {required_cols}")

# Apply filters based on time period
valid_periods = ['last_month', 'last_year', 'all_time']
if time_period not in valid_periods:
    raise ValueError(f"time_period must be one of {valid_periods}")

# Filter by revenue threshold
return df[df['revenue'] > revenue_threshold]

print("Example function with comprehensive docstring (Google style):")
print(filter_high_value_customers.__doc__)

print("\n📝 DOCSTRING STYLES:")
print("""
1. GOOGLE STYLE (recommended for analytics):
    Args:
        param_name: Description
    Returns:
        Description

2. NUMPY STYLE (for scientific computing):
    Parameters
    -----
    param_name : type
        Description

3. SPHINX STYLE:
    :param param_name: Description
    :type param_name: type
    :returns: Description
""")

print("\n💡 WHEN TO COMMENT:")
print("    ✓ WHY, not WHAT:")
print("        # Use log transformation to normalize skewed distribution")
print("        df['log_sales'] = np.log1p(df['sales'])")
print("")
print("    ✓ Complex algorithms:")
print("        # Implementing Smith-Waterman algorithm for sequence alignment")
print("")
print("    ✓ Business logic:")
print("        # Discount applies only to customers with 3+ years tenure (policy)")
print("")
print("    ✗ DON'T comment obvious code:")
print("        total = price * quantity # No comment needed!")

print("\n🔧 DOCSTRING TOOLS:")
print("    • pydoc: Built-in (help(function))")
print("    • Sphinx: Auto-generate documentation")
print("    • pdoc: Simple auto-docs")

```

```
print("    • VS Code: Auto-complete from docstrings")  
print("="*70)
```

PRACTICE #2: DOCUMENTATION

=====

❌ BAD DOCUMENTATION:

```
# Bad: Obvious comments (noise)
x = x + 1 # Increment x

# Bad: Outdated comments
# Calculate total (but actually calculates average now)
avg = sum(values) / len(values)

# Bad: No documentation
def process(df, threshold, mode):
    # What does this do? What are valid modes?
    pass
```

✅ GOOD DOCUMENTATION:

Example function with comprehensive docstring (Google style):

Filter customers with revenue above threshold in specified period.

Args:

df: DataFrame with columns ['customer_id', 'revenue', 'date']
revenue_threshold: Minimum revenue to qualify (default: 10000.0)
time_period: One of ['last_month', 'last_year', 'all_time']

Returns:

Filtered DataFrame with high-value customers

Raises:

ValueError: If time_period is invalid
KeyError: If required columns missing

Example:

```
>>> high_value = filter_high_value_customers(
...     df, revenue_threshold=50000, time_period='last_year'
... )
```

DOCSTRING STYLES:

1. GOOGLE STYLE (recommended for analytics):

Args:

param_name: Description

Returns:

Description

2. NUMPY STYLE (for scientific computing):

Parameters

param_name : type

Description

3. SPHINX STYLE:

```
:param param_name: Description
:type param_name: type
:returns: Description
```

💡 WHEN TO COMMENT:

✓ WHY, not WHAT:

```
# Use log transformation to normalize skewed distribution
df['log_sales'] = np.log1p(df['sales'])
```

✓ Complex algorithms:

```
# Implementing Smith-Waterman algorithm for sequence alignment
```

✓ Business logic:

```
# Discount applies only to customers with 3+ years tenure (policy #23
```

4)

✗ DON'T comment obvious code:

```
total = price * quantity # No comment needed!
```

✂️ DOCSTRING TOOLS:

- pydoc: Built-in (help(function))
- Sphinx: Auto-generate documentation
- pdoc: Simple auto-docs
- VS Code: Auto-complete from docstrings

=====

3. Practice #3: Type Hints - Catch Bugs Before Runtime

```
In [5]: print("\nPRACTICE #3: TYPE HINTS")
print("="*70)

print("\n❌ WITHOUT TYPE HINTS:")
print("""
def calculate_discount(price, discount_pct, customer_type):
    # What types? No idea until runtime error!
    return price * (1 - discount_pct)

# This crashes at runtime
calculate_discount("100", "0.1", None) # Type error!
""")

print("\n✅ WITH TYPE HINTS:")
print("""

def calculate_discount(
    price: float,
    discount_pct: float,
    customer_type: str = 'regular'
) -> float:
    """
    Calculate discounted price.
```

```

    Args:
        price: Original price (must be positive)
        discount_pct: Discount as decimal (0.1 = 10%)
        customer_type: 'regular', 'premium', or 'vip'

    Returns:
        Final price after discount
    """
    if not 0 <= discount_pct <= 1:
        raise ValueError("Discount must be between 0 and 1")
    return price * (1 - discount_pct)

print("def calculate_discount(price: float, discount_pct: float, ")
print("                                customer_type: str = 'regular') -> float:")
print("    # Type checker (mypy) catches errors before running!")

print("\n🔗 COMMON TYPE HINTS:")
print("""
# Basic types
age: int = 25
price: float = 99.99
name: str = "Alice"
is_active: bool = True

# Collections
scores: List[int] = [85, 90, 78]
user_data: Dict[str, int] = {'age': 25, 'score': 100}
coordinates: Tuple[float, float] = (10.5, 20.3)
unique_ids: Set[int] = {1, 2, 3}

# Optional (can be None)
middle_name: Optional[str] = None
error_msg: Optional[str] = "Error occurred"

# Union (multiple types)
id_value: Union[int, str] = "USER123"

# For DataFrames (common in analytics)
def process_data(df: pd.DataFrame) -> pd.DataFrame:
    return df

# Callable (function as argument)
from typing import Callable
def apply_transform(data: List[int],
                    func: Callable[[int], int]) -> List[int]:
    return [func(x) for x in data]
""")

# Practical example
def aggregate_sales(
    df: pd.DataFrame,
    group_by: List[str],
    agg_func: str = 'sum'
) -> pd.DataFrame:
    """

```

Aggregate sales data by specified columns.

Args:

df: Sales DataFrame
group_by: List of column names to group by
agg_func: Aggregation function ('sum', 'mean', 'count')

Returns:

Aggregated DataFrame

"""

return df.groupby(group_by).agg(agg_func)

Create sample data

```
sample_df = pd.DataFrame({  
    'product': ['A', 'B', 'A', 'B'],  
    'region': ['North', 'North', 'South', 'South'],  
    'sales': [100, 200, 150, 250]  
})
```

```
result = aggregate_sales(sample_df, ['product'], 'sum')
```

```
print("\n📊 Example with type hints:")
```

```
print(result)
```

```
print("\n💡 PRO TIPS:")
```

```
print("    1. Use mypy for static type checking: 'mypy script.py'")
```

```
print("    2. Start with function signatures, then add more")
```

```
print("    3. Type hints improve IDE autocomplete (huge productivity boost!)")
```

```
print("    4. Use Optional[T] instead of Union[T, None]")
```

```
print("    5. For complex types, use TypeAlias (Python 3.10+):")
```

```
print("        UserData = Dict[str, Union[int, str, List[str]]]")
```

```
print("\n⚡ TYPE CHECKING IN CI/CD:")
```

```
print("    # In your CI pipeline")
```

```
print("    pip install mypy")
```

```
print("    mypy --strict src/")
```

```
print("    # Catches type errors before deployment!")
```

```
print("=*70)
```

PRACTICE #3: TYPE HINTS

✗ WITHOUT TYPE HINTS:

```
def calculate_discount(price, discount_pct, customer_type):
    # What types? No idea until runtime error!
    return price * (1 - discount_pct)
```

```
# This crashes at runtime
calculate_discount("100", "0.1", None) # Type error!
```

✓ WITH TYPE HINTS:

```
def calculate_discount(price: float, discount_pct: float,
                       customer_type: str = 'regular') -> float:
    # Type checker (mypy) catches errors before running!
```

🌀 COMMON TYPE HINTS:

```
# Basic types
age: int = 25
price: float = 99.99
name: str = "Alice"
is_active: bool = True
```

```
# Collections
scores: List[int] = [85, 90, 78]
user_data: Dict[str, int] = {'age': 25, 'score': 100}
coordinates: Tuple[float, float] = (10.5, 20.3)
unique_ids: Set[int] = {1, 2, 3}
```

```
# Optional (can be None)
middle_name: Optional[str] = None
error_msg: Optional[str] = "Error occurred"
```

```
# Union (multiple types)
id_value: Union[int, str] = "USER123"
```

```
# For DataFrames (common in analytics)
def process_data(df: pd.DataFrame) -> pd.DataFrame:
    return df
```

```
# Callable (function as argument)
from typing import Callable
def apply_transform(data: List[int],
                    func: Callable[[int], int]) -> List[int]:
    return [func(x) for x in data]
```

📊 Example with type hints:

	region	sales
product		
A	NorthSouth	250
B	NorthSouth	450

💡 PRO TIPS:

1. Use mypy for static type checking: 'mypy script.py'
2. Start with function signatures, then add more
3. Type hints improve IDE autocomplete (huge productivity boost!)
4. Use Optional[T] instead of Union[T, None]
5. For complex types, use TypeAlias (Python 3.10+):
 UserData = Dict[str, Union[int, str, List[str]]]

✂ TYPE CHECKING IN CI/CD:

```
# In your CI pipeline
pip install mypy
mypy --strict src/
# Catches type errors before deployment!
```

=====

4. Practice #4: Function Design - Single Responsibility Principle

```
In [6]: print("\nPRACTICE #4: FUNCTION DESIGN - SINGLE RESPONSIBILITY")
print("="*70)

print("\n❌ BAD: God Function (does everything)")
print("""
def process_sales_data(filepath):
    # Does 10 different things!
    df = pd.read_csv(filepath)
    df = df.dropna()
    df['revenue'] = df['price'] * df['quantity']
    df = df[df['revenue'] > 0]
    df = df.groupby('category').agg({'revenue': 'sum'})
    df = df.sort_values('revenue', ascending=False)
    df.to_csv('output.csv')
    print("Processing complete!")
    # Hard to test, hard to reuse, hard to understand
    return df
""")

print("\n✅ GOOD: Single Responsibility (one job per function)")
print("")

def load_sales_data(filepath: str) -> pd.DataFrame:
    """Load sales data from CSV file."""
    return pd.read_csv(filepath)

def clean_sales_data(df: pd.DataFrame) -> pd.DataFrame:
    """Remove rows with missing values."""
    return df.dropna()

def calculate_revenue(df: pd.DataFrame) -> pd.DataFrame:
    """Calculate revenue as price × quantity."""
    df = df.copy()
    df['revenue'] = df['price'] * df['quantity']
    return df
```

```

def filter_positive_revenue(df: pd.DataFrame) -> pd.DataFrame:
    """Keep only rows with positive revenue."""
    return df[df['revenue'] > 0]

def aggregate_by_category(df: pd.DataFrame) -> pd.DataFrame:
    """Aggregate revenue by category."""
    return df.groupby('category').agg({'revenue': 'sum'})

def sort_by_revenue(df: pd.DataFrame, ascending: bool = False) -> pd.DataFrame:
    """Sort by revenue."""
    return df.sort_values('revenue', ascending=ascending)

def save_results(df: pd.DataFrame, filepath: str) -> None:
    """Save DataFrame to CSV."""
    df.to_csv(filepath, index=False)
    print(f"✓ Saved to {filepath}")

# Now compose them in a pipeline
def process_sales_pipeline(input_path: str, output_path: str) -> pd.DataFrame:
    """
    Complete sales processing pipeline.

    Pipeline steps:
    1. Load data
    2. Clean (remove nulls)
    3. Calculate revenue
    4. Filter positive values
    5. Aggregate by category
    6. Sort by revenue
    7. Save results
    """
    df = (load_sales_data(input_path)
          .pipe(clean_sales_data)
          .pipe(calculate_revenue)
          .pipe(filter_positive_revenue)
          .pipe(aggregate_by_category)
          .pipe(sort_by_revenue))

    save_results(df, output_path)
    return df

print("Benefits of single responsibility:")
print("    ✓ Each function is testable independently")
print("    ✓ Reusable in other contexts")
print("    ✓ Easy to understand and maintain")
print("    ✓ Can modify one step without affecting others")

print("\n💡 FUNCTION DESIGN RULES:")
print("    1. One function = One responsibility")
print("    2. Keep functions short (<50 lines, ideally <20)")
print("    3. Function name should describe what it does")
print("    4. Limit parameters (max 3-5, use dataclass for more)")
print("    5. Avoid side effects (don't modify inputs)")
print("    6. Return early (guard clauses at top)")

```

```

print("\n🌟 PRACTICAL EXAMPLE:")

# Create sample data
sample_sales = pd.DataFrame({
    'category': ['Electronics', 'Books', 'Electronics', 'Books', 'Clothing'],
    'price': [100, 20, 150, 25, 50],
    'quantity': [2, 5, 1, 3, 4]
})

# Process step by step
print("\nStep-by-step processing:")
df_with_revenue = calculate_revenue(sample_sales)
print("\n1. After calculating revenue:")
print(df_with_revenue)

df_aggregated = aggregate_by_category(df_with_revenue)
print("\n2. After aggregating by category:")
print(df_aggregated)

df_sorted = sort_by_revenue(df_aggregated)
print("\n3. After sorting:")
print(df_sorted)

print("\n✓ Each step is clear, testable, and reusable!")
print("=*70)

```

PRACTICE #4: FUNCTION DESIGN – SINGLE RESPONSIBILITY

❌ BAD: God Function (does everything)

```
def process_sales_data(filepath):
    # Does 10 different things!
    df = pd.read_csv(filepath)
    df = df.dropna()
    df['revenue'] = df['price'] * df['quantity']
    df = df[df['revenue'] > 0]
    df = df.groupby('category').agg({'revenue': 'sum'})
    df = df.sort_values('revenue', ascending=False)
    df.to_csv('output.csv')
    print("Processing complete!")
    # Hard to test, hard to reuse, hard to understand
    return df
```

✅ GOOD: Single Responsibility (one job per function)

Benefits of single responsibility:

- ✓ Each function is testable independently
- ✓ Reusable in other contexts
- ✓ Easy to understand and maintain
- ✓ Can modify one step without affecting others

💡 FUNCTION DESIGN RULES:

1. One function = One responsibility
2. Keep functions short (<50 lines, ideally <20)
3. Function name should describe what it does
4. Limit parameters (max 3–5, use dataclass for more)
5. Avoid side effects (don't modify inputs)
6. Return early (guard clauses at top)

🌀 PRACTICAL EXAMPLE:

Step-by-step processing:

1. After calculating revenue:

	category	price	quantity	revenue
0	Electronics	100	2	200
1	Books	20	5	100
2	Electronics	150	1	150
3	Books	25	3	75
4	Clothing	50	4	200

2. After aggregating by category:

	revenue
category	
Books	175
Clothing	200
Electronics	350

3. After sorting:

	revenue
--	---------

category	
Electronics	350
Clothing	200
Books	175

✓ Each step is clear, testable, and reusable!

=====

5. Practice #5: List Comprehensions - Pythonic and Fast

```
In [7]: print("\nPRACTICE #5: LIST COMPREHENSIONS")
print("="*70)

print("\n✗ BAD: Explicit loops (slow and verbose)")
print("""
# Transform data
squares = []
for x in range(10):
    squares.append(x ** 2)

# Filter data
evens = []
for x in range(10):
    if x % 2 == 0:
        evens.append(x)

# Nested loops
pairs = []
for x in range(3):
    for y in range(3):
        pairs.append((x, y))
""")

print("\n✓ GOOD: List comprehensions (fast and readable)")
print("""

# Transform
squares = [x**2 for x in range(10)]
print("\n1. Transform: [x**2 for x in range(10)]")
print(f"    Result: {squares}")

# Filter
evens = [x for x in range(10) if x % 2 == 0]
print("\n2. Filter: [x for x in range(10) if x % 2 == 0]")
print(f"    Result: {evens}")

# Transform + Filter
even_squares = [x**2 for x in range(10) if x % 2 == 0]
print("\n3. Transform + Filter: [x**2 for x in range(10) if x % 2 == 0]")
print(f"    Result: {even_squares}")

# Nested comprehension
pairs = [(x, y) for x in range(3) for y in range(3)]
```

```

print("\n4. Nested: [(x, y) for x in range(3) for y in range(3)]")
print(f"    Result: {pairs}")

# Dictionary comprehension
squares_dict = {x: x**2 for x in range(5)}
print("\n5. Dict comprehension: {x: x**2 for x in range(5)}")
print(f"    Result: {squares_dict}")

# Set comprehension
unique_lengths = {len(word) for word in ['hello', 'world', 'hi', 'bye']}
print("\n6. Set comprehension: {len(word) for word in words}")
print(f"    Result: {unique_lengths}")

print("\n🌟 ANALYTICS EXAMPLES:")

# Sample data
sales_data = [
    {'product': 'Laptop', 'price': 1000, 'quantity': 2},
    {'product': 'Mouse', 'price': 20, 'quantity': 10},
    {'product': 'Keyboard', 'price': 50, 'quantity': 5}
]

# Extract prices
prices = [item['price'] for item in sales_data]
print("\n1. Extract prices:")
print(f"    {prices}")

# Calculate revenues
revenues = [item['price'] * item['quantity'] for item in sales_data]
print("\n2. Calculate revenues:")
print(f"    {revenues}")

# Filter high-value items
high_value = [item['product'] for item in sales_data
               if item['price'] * item['quantity'] > 100]
print("\n3. High-value products (revenue > 100):")
print(f"    {high_value}")

# Create lookup dictionary
price_lookup = {item['product']: item['price'] for item in sales_data}
print("\n4. Price lookup dictionary:")
print(f"    {price_lookup}")

print("\n⚡ PERFORMANCE COMPARISON:")
import time

# Loop version
start = time.time()
result1 = []
for i in range(100000):
    result1.append(i ** 2)
loop_time = time.time() - start

# List comprehension
start = time.time()
result2 = [i ** 2 for i in range(100000)]

```

```
comp_time = time.time() - start

print(f"    Loop: {loop_time:.4f}s")
print(f"    List comprehension: {comp_time:.4f}s")
print(f"    Speedup: {loop_time/comp_time:.2f}x faster!")

print("\n💡 PRO TIPS:")
print("    1. Keep comprehensions simple (max 2 lines)")
print("    2. Complex logic? Use regular loop for readability")
print("    3. Generator expressions for large data: (x**2 for x in range(10))")
print("    4. Use map() for function application: map(str.upper, words)")
print("    5. Use filter() for simple filtering: filter(lambda x: x > 0, nums)")

print("\n⚠️ WHEN NOT TO USE:")
print("    ❌ Complex nested logic (use regular loops)")
print("    ❌ Side effects (print, file writes)")
print("    ❌ When readability suffers")
print("\n="*70)
```

PRACTICE #5: LIST COMPREHENSIONS

❌ BAD: Explicit loops (slow and verbose)

```
# Transform data
squares = []
for x in range(10):
    squares.append(x ** 2)

# Filter data
evens = []
for x in range(10):
    if x % 2 == 0:
        evens.append(x)

# Nested loops
pairs = []
for x in range(3):
    for y in range(3):
        pairs.append((x, y))
```

✅ GOOD: List comprehensions (fast and readable)

1. Transform: `[x**2 for x in range(10)]`
Result: `[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]`
2. Filter: `[x for x in range(10) if x % 2 == 0]`
Result: `[0, 2, 4, 6, 8]`
3. Transform + Filter: `[x**2 for x in range(10) if x % 2 == 0]`
Result: `[0, 4, 16, 36, 64]`
4. Nested: `[(x, y) for x in range(3) for y in range(3)]`
Result: `[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)]`
5. Dict comprehension: `{x: x**2 for x in range(5)}`
Result: `{0: 0, 1: 1, 2: 4, 3: 9, 4: 16}`
6. Set comprehension: `{len(word) for word in words}`
Result: `{2, 3, 5}`

🎯 ANALYTICS EXAMPLES:

1. Extract prices:
`[1000, 20, 50]`
2. Calculate revenues:
`[2000, 200, 250]`
3. High-value products (revenue > 100):
`['Laptop', 'Mouse', 'Keyboard']`
4. Price lookup dictionary:

```
{'Laptop': 1000, 'Mouse': 20, 'Keyboard': 50}
```

✗ PERFORMANCE COMPARISON:

Loop: 0.0053s

List comprehension: 0.0024s

Speedup: 2.17x faster!

💡 PRO TIPS:

1. Keep comprehensions simple (max 2 lines)
2. Complex logic? Use regular loop for readability
3. Generator expressions for large data: `(x**2 for x in range(10))`
4. Use `map()` for function application: `map(str.upper, words)`
5. Use `filter()` for simple filtering: `filter(lambda x: x > 0, nums)`

⚠️ WHEN NOT TO USE:

- ✗ Complex nested logic (use regular loops)
- ✗ Side effects (print, file writes)
- ✗ When readability suffers

=====

6. Practice #6: Error Handling - Fail Gracefully

```
In [8]: print("\nPRACTICE #6: ERROR HANDLING")
        print("=*70)

        print("\n✗ BAD: No error handling (silent failures or crashes)")
        print("""
        # Crashes on bad input
        def calculate_average(numbers):
            return sum(numbers) / len(numbers) # ZeroDivisionError if empty!

        # Silent failure (returns None)
        def get_user_age(user_id):
            user = database.get(user_id) # What if not found?
            return user['age'] # KeyError or AttributeError!
        """)

        print("\n✅ GOOD: Proper error handling")
        print("")

        def calculate_average(numbers: List[float]) -> Optional[float]:
            """
            Calculate average of numbers.

            Args:
                numbers: List of numbers

            Returns:
                Average value, or None if list is empty

            Raises:
                TypeError: If numbers contains non-numeric values
            """
            if not numbers:
                return None
```

```

    try:
        return sum(numbers) / len(numbers)
    except TypeError as e:
        raise TypeError(f"All elements must be numeric: {e}")

# Test cases
print("1. Normal case:")
result = calculate_average([10, 20, 30])
print(f"    Average of [10, 20, 30] = {result}")

print("\n2. Empty list:")
result = calculate_average([])
print(f"    Average of [] = {result}")

print("\n3. Error case:")
try:
    result = calculate_average([10, 'twenty', 30])
except TypeError as e:
    print(f"    ✓ Caught error: {e}")

print("\n🌟 PRACTICAL EXAMPLE: Data Loading")

def load_sales_data_safe(filepath: str) -> Optional[pd.DataFrame]:
    """
    Load sales data with comprehensive error handling.

    Args:
        filepath: Path to CSV file

    Returns:
        DataFrame if successful, None otherwise
    """
    try:
        # Check file exists
        if not Path(filepath).exists():
            print(f"❌ File not found: {filepath}")
            return None

        # Load data
        df = pd.read_csv(filepath)

        # Validate required columns
        required_cols = ['product', 'price', 'quantity']
        missing_cols = [col for col in required_cols if col not in df.columns]
        if missing_cols:
            print(f"❌ Missing columns: {missing_cols}")
            return None

        print(f"✓ Successfully loaded {len(df)} rows from {filepath}")
        return df

    except pd.errors.EmptyDataError:
        print(f"❌ File is empty: {filepath}")
        return None
    except pd.errors.ParserError as e:

```

```

        print(f"❌ Failed to parse CSV: {e}")
        return None
    except Exception as e:
        print(f"❌ Unexpected error: {type(e).__name__}: {e}")
        return None

print("\nExample: Loading non-existent file")
df = load_sales_data_safe('non_existent.csv')
print(f"Result: {df}")

print("\n💡 ERROR HANDLING PATTERNS:")
print("""
1. EAFP (Easier to Ask Forgiveness than Permission):
    try:
        value = data['key']
    except KeyError:
        value = default

2. LBYL (Look Before You Leap):
    if 'key' in data:
        value = data['key']
    else:
        value = default

3. Context Managers (auto cleanup):
    with open('file.txt') as f:
        data = f.read()
    # File auto-closed even if error!
""")

print("\n📁 EXCEPTION HIERARCHY (catch specific exceptions):")
print("""
BaseException
├── Exception
│   ├── ValueError (invalid value)
│   ├── TypeError (wrong type)
│   ├── KeyError (dict key not found)
│   ├── IndexError (list index out of range)
│   ├── FileNotFoundError (file doesn't exist)
│   └── ZeroDivisionError
└── KeyboardInterrupt (don't catch this!)
""")

print("\n💡 PRO TIPS:")
print("    1. Catch specific exceptions, not bare 'except:'.")
print("    2. Fail fast – validate inputs early")
print("    3. Log errors for debugging (use logging module)")
print("    4. Use custom exceptions for domain logic")
print("    5. Don't silence errors – handle or raise")
print("    6. Use 'finally' for cleanup (close files, connections)")
print("    7. Use '*' for wildcard")

```

PRACTICE #6: ERROR HANDLING

❌ BAD: No error handling (silent failures or crashes)

```
# Crashes on bad input
def calculate_average(numbers):
    return sum(numbers) / len(numbers) # ZeroDivisionError if empty!

# Silent failure (returns None)
def get_user_age(user_id):
    user = database.get(user_id) # What if not found?
    return user['age'] # KeyError or AttributeError!
```

✅ GOOD: Proper error handling

1. Normal case:
Average of [10, 20, 30] = 20.0
2. Empty list:
Average of [] = None
3. Error case:
✓ Caught error: All elements must be numeric: unsupported operand type(s) for +: 'int' and 'str'

🔗 PRACTICAL EXAMPLE: Data Loading

Example: Loading non-existent file

❌ File not found: non_existent.csv

Result: None

💡 ERROR HANDLING PATTERNS:

1. EAFP (Easier to Ask Forgiveness than Permission):

```
try:
    value = data['key']
except KeyError:
    value = default
```

2. LBYL (Look Before You Leap):

```
if 'key' in data:
    value = data['key']
else:
    value = default
```

3. Context Managers (auto cleanup):

```
with open('file.txt') as f:
    data = f.read()
# File auto-closed even if error!
```

📁 EXCEPTION HIERARCHY (catch specific exceptions):

BaseException

```

├── Exception
│   ├── ValueError (invalid value)
│   ├── TypeError (wrong type)
│   ├── KeyError (dict key not found)
│   ├── IndexError (list index out of range)
│   ├── FileNotFoundError (file doesn't exist)
│   └── ZeroDivisionError
└── KeyboardInterrupt (don't catch this!)

```



PRO TIPS:

1. Catch specific exceptions, not bare 'except:'
2. Fail fast – validate inputs early
3. Log errors for debugging (use logging module)
4. Use custom exceptions for domain logic
5. Don't silence errors – handle or raise
6. Use 'finally' for cleanup (close files, connections)

=====

7. Practice #7: DRY Principle - Don't Repeat Yourself

```

In [9]: print("\nPRACTICE #7: DRY PRINCIPLE – DON'T REPEAT YOURSELF")
        print("=*70)

        print("\n❌ BAD: Code duplication (WET – Write Everything Twice)")
        print("""
# Calculate metrics for different regions – repeated code!
north_sales = df[df['region'] == 'North']['sales'].sum()
north_avg = df[df['region'] == 'North']['sales'].mean()
north_count = len(df[df['region'] == 'North'])

south_sales = df[df['region'] == 'South']['sales'].sum()
south_avg = df[df['region'] == 'South']['sales'].mean()
south_count = len(df[df['region'] == 'South'])

east_sales = df[df['region'] == 'East']['sales'].sum()
east_avg = df[df['region'] == 'East']['sales'].mean()
east_count = len(df[df['region'] == 'East'])

# 15 lines! What if logic changes? Update 3 places!
""")

        print("\n✅ GOOD: DRY – Extract common logic")
        print("""

def calculate_region_metrics(
    df: pd.DataFrame,
    region: str
) -> Dict[str, float]:
    """
    Calculate sales metrics for a specific region.

    Args:
        df: Sales DataFrame
        region: Region name

```

```

Returns:
    Dictionary with total, average, and count
"""
region_data = df[df['region'] == region]['sales']
return {
    'total': region_data.sum(),
    'average': region_data.mean(),
    'count': len(region_data)
}

# Now use it for all regions
sample_df = pd.DataFrame({
    'region': ['North', 'South', 'East', 'North', 'South'],
    'sales': [100, 200, 150, 120, 180]
})

regions = ['North', 'South', 'East']
metrics = {region: calculate_region_metrics(sample_df, region)
            for region in regions}

print("Regional metrics:")
for region, metric in metrics.items():
    print(f"\n{region}:")
    print(f"    Total: ${metric['total']:.2f}")
    print(f"    Average: ${metric['average']:.2f}")
    print(f"    Count: {metric['count']}")

print("\n\n☺ MORE DRY EXAMPLES:")

# Bad: Magic numbers repeated
print("\n❌ BAD: Magic numbers")
print("""
if price > 1000:
    discount = 0.15
elif price > 500:
    discount = 0.10
# What if thresholds change? Search and replace everywhere!
""")

# Good: Constants at top
print("\n✅ GOOD: Named constants")
PREMIUM_THRESHOLD = 1000
STANDARD_THRESHOLD = 500
PREMIUM_DISCOUNT = 0.15
STANDARD_DISCOUNT = 0.10

def calculate_discount(price: float) -> float:
    """Calculate discount based on price tier."""
    if price > PREMIUM_THRESHOLD:
        return PREMIUM_DISCOUNT
    elif price > STANDARD_THRESHOLD:
        return STANDARD_DISCOUNT
    return 0.0

print("def calculate_discount(price: float) -> float:")

```

```

print("    if price > PREMIUM_THRESHOLD:")
print("        return PREMIUM_DISCOUNT")
print("    # Easy to update – change constant once!")

# Bad: Repeated validation
print("\n❌ BAD: Repeated validation logic")
print("""
def process_order_1(order_id):
    if order_id is None or order_id < 0:
        raise ValueError("Invalid order ID")
    # ... process

def process_order_2(order_id):
    if order_id is None or order_id < 0: # Duplicated!
        raise ValueError("Invalid order ID")
    # ... process
""")

# Good: Decorator for validation
print("\n✅ GOOD: Decorator pattern")
from functools import wraps

def validate_order_id(func):
    """Decorator to validate order_id parameter."""
    @wraps(func)
    def wrapper(order_id: int, *args, **kwargs):
        if order_id is None or order_id < 0:
            raise ValueError(f"Invalid order ID: {order_id}")
        return func(order_id, *args, **kwargs)
    return wrapper

@validate_order_id
def process_order_1(order_id: int) -> str:
    return f"Processing order {order_id}"

@validate_order_id
def process_order_2(order_id: int) -> str:
    return f"Shipping order {order_id}"

print("@validate_order_id")
print("def process_order(order_id: int):")
print("    # Validation automatic!")

# Test
print("\nTest valid order:")
print(f"    {process_order_1(123)}")

print("\nTest invalid order:")
try:
    process_order_1(-1)
except ValueError as e:
    print(f"    ✓ Caught: {e}")

print("\n💡 DRY PRINCIPLES:")
print("    1. Extract common logic into functions")
print("    2. Use constants instead of magic numbers")

```

```
print("    3. Create utility modules for repeated operations")
print("    4. Use decorators for cross-cutting concerns")
print("    5. Configuration files for repeated values")

print("\n⚠ BUT DON'T OVER-DRY:")
print("    • Duplication is better than wrong abstraction")
print("    • Rule of three: Abstract after 3rd repetition")
print("    • Keep it simple – don't create complex abstractions")
print("=*70)
```

PRACTICE #7: DRY PRINCIPLE – DON'T REPEAT YOURSELF

❌ BAD: Code duplication (WET – Write Everything Twice)

```
# Calculate metrics for different regions – repeated code!
north_sales = df[df['region'] == 'North']['sales'].sum()
north_avg = df[df['region'] == 'North']['sales'].mean()
north_count = len(df[df['region'] == 'North'])

south_sales = df[df['region'] == 'South']['sales'].sum()
south_avg = df[df['region'] == 'South']['sales'].mean()
south_count = len(df[df['region'] == 'South'])

east_sales = df[df['region'] == 'East']['sales'].sum()
east_avg = df[df['region'] == 'East']['sales'].mean()
east_count = len(df[df['region'] == 'East'])

# 15 lines! What if logic changes? Update 3 places!
```

✅ GOOD: DRY – Extract common logic

Regional metrics:

North:

Total: \$220.00
Average: \$110.00
Count: 2

South:

Total: \$380.00
Average: \$190.00
Count: 2

East:

Total: \$150.00
Average: \$150.00
Count: 1

🌀 MORE DRY EXAMPLES:

❌ BAD: Magic numbers

```
if price > 1000:
    discount = 0.15
elif price > 500:
    discount = 0.10
# What if thresholds change? Search and replace everywhere!
```

✅ GOOD: Named constants

```
def calculate_discount(price: float) -> float:
    if price > PREMIUM_THRESHOLD:
        return PREMIUM_DISCOUNT
```

Easy to update – change constant once!

✗ BAD: Repeated validation logic

```
def process_order_1(order_id):
    if order_id is None or order_id < 0:
        raise ValueError("Invalid order ID")
    # ... process

def process_order_2(order_id):
    if order_id is None or order_id < 0: # Duplicated!
        raise ValueError("Invalid order ID")
    # ... process
```

✓ GOOD: Decorator pattern

```
@validate_order_id
def process_order(order_id: int):
    # Validation automatic!
```

Test valid order:
Processing order 123

Test invalid order:
✓ Caught: Invalid order ID: -1

💡 DRY PRINCIPLES:

1. Extract common logic into functions
2. Use constants instead of magic numbers
3. Create utility modules for repeated operations
4. Use decorators for cross-cutting concerns
5. Configuration files for repeated values

⚠ BUT DON'T OVER-DRY:

- Duplication is better than wrong abstraction
- Rule of three: Abstract after 3rd repetition
- Keep it simple – don't create complex abstractions

=====

Key Takeaways

```
In [10]: print("\n" + "="*70)
print("KEY TAKEAWAYS – CLEAN PYTHON FOR ANALYTICS")
print("="*70)

takeaways = """
🎯 7 ESSENTIAL PRACTICES:

1. NAMING CONVENTIONS (PEP 8)
    • Variables/functions: snake_case
    • Classes: PascalCase
    • Constants: UPPER_CASE
    • Use descriptive names: calculate_revenue > calc()
    • Booleans: is_, has_, can_
```

2. DOCUMENTATION

- Docstrings for all functions (Google/NumPy style)
- Comments explain WHY, not WHAT
- Use `help(function)` to test docstrings
- Tools: `pydoc`, `Sphinx`, `pdoc`

3. TYPE HINTS

- Add types to function signatures
- Use `mypy` for static type checking
- Improves IDE autocomplete
- Catches bugs before runtime
- `Optional[T]`, `Union[T1, T2]`, `List[T]`

4. FUNCTION DESIGN

- Single Responsibility Principle
- Keep functions short (<20 lines ideal)
- Limit parameters (max 3-5)
- Avoid side effects
- Use `.pipe()` for pandas chains

5. LIST COMPREHENSIONS

- `[x**2 for x in range(10)]`
- 30-50% faster than loops
- More Pythonic and readable
- Use generators for large data: `(x for x in ...)`
- Dict/set comprehensions too

6. ERROR HANDLING

- Catch specific exceptions, not bare `except`
- Fail fast – validate inputs early
- Use `try/except/finally`
- Return `None` or `raise`, don't return error codes
- Log errors for debugging

7. DRY PRINCIPLE

- Don't Repeat Yourself
- Extract common logic into functions
- Use constants for magic numbers
- Decorators for cross-cutting concerns
- Rule of three: abstract after 3rd repetition



QUICK WINS:

```
# Before: Messy
def f(d, x):
    return d[d[x] > 100]

# After: Clean
def filter_high_value_sales(
    sales_df: pd.DataFrame,
    threshold: float = 100.0
) -> pd.DataFrame:
    """Filter sales above threshold."""
    return sales_df[sales_df['amount'] > threshold]
```



TOOLS TO USE:

- black: Auto-format code
- mypy: Static type checking
- pylint: Linting and code quality
- isort: Sort imports
- pre-commit: Git hooks for quality

✂ REMEMBER:

"Code is read 10x more than it's written.
Write for humans, not machines."

"Clean code is not about being fancy.
It's about being maintainable."

"Your future self will thank you
(or curse you)."

📖 FURTHER READING:

- Clean Code by Robert Martin
- PEP 8 – Python Style Guide
- PEP 257 – Docstring Conventions
- The Zen of Python (import this)

"""

```
print(takeaways)
print("="*70)
```

Bonus: The Zen of Python

```
print("\n🐍 BONUS: THE ZEN OF PYTHON")
import this
```

Cell In[10], line 67

```
"""Filter sales above threshold."""
    ^
```

SyntaxError: invalid syntax